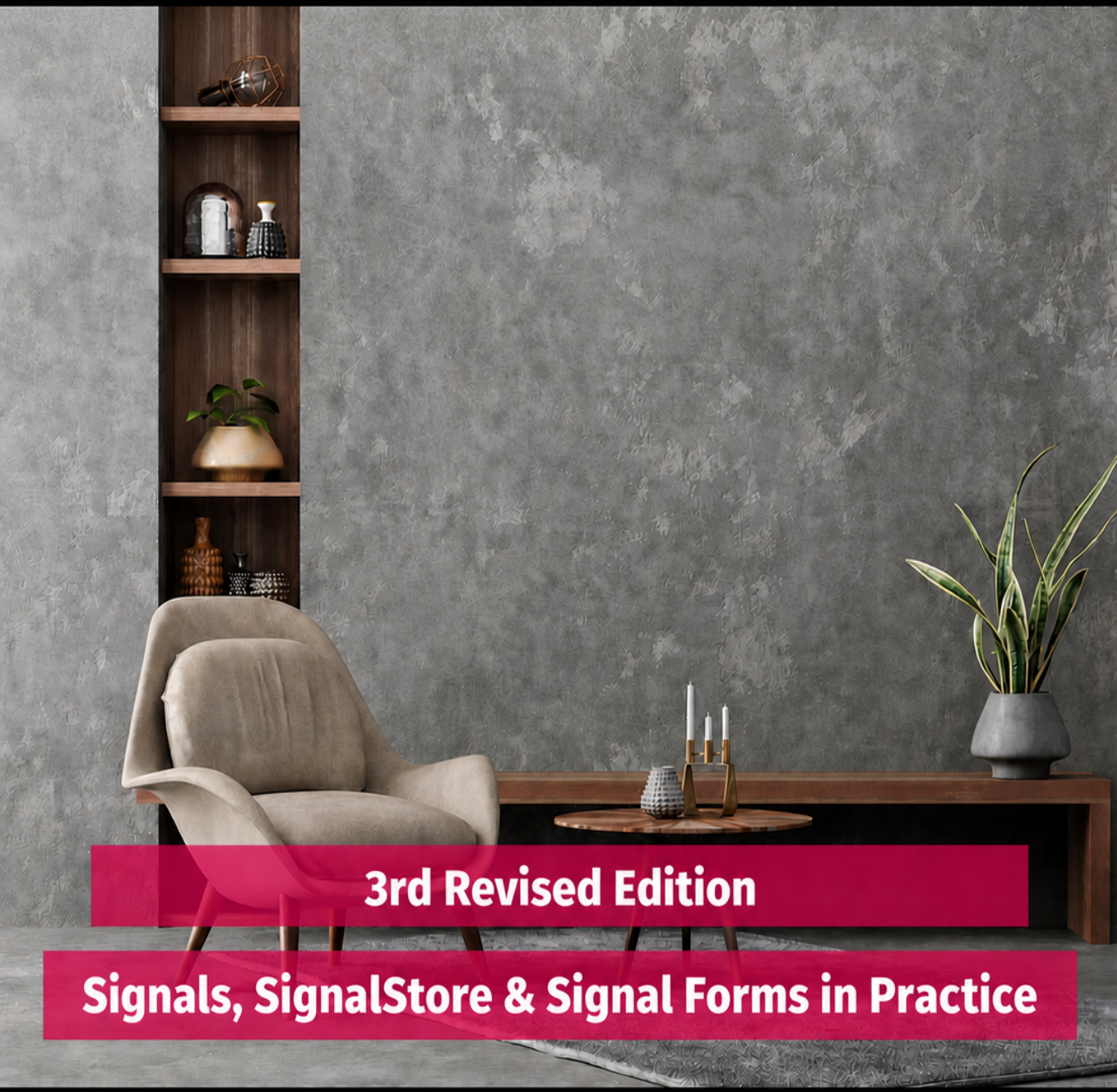
Angular Signals Fast Track

ANGULAR ARCHITECTS



3rd Revised Edition

Signals, SignalStore & Signal Forms in Practice

MANFRED STEYER

Angular Signals Fast Track

Signals, SignalStore und Signal Forms in Practice

Manfred Steyer

3rd Edition, May 2, 2026

Contents

Preface **1**
Who This Book Is For . . . 1
Source Code . . . 1
Feedback . . . 2
About the Author . . . 2
Trainings and Consultancy . . . 2

1 Reactive Design with Signals **4**
Building Blocks of Reactive Design . . . 4
Computed Signals . . . 4
Resources . . . 6
httpResource . . . 6
rxResource . . . 8
resource (Promise-based) . . . 10
Effects . . . 12
Signal Semantics in Angular . . . 15
Signals in the Component Lifecycle . . . 15
Auto-Tracking and the Reactive Context . . . 16
Discussing Explicit Effects . . . 18
Untracking . . . 19
Glitch-Free Property . . . 19
Equality and Immutability . . . 20
Establishing a Reactive Flow . . . 22
Thinking in Terms of the Signal Graph . . . 22
Implementing the Reactive Flow . . . 23
Summary . . . 26

2 Signal Forms **27**
A First Signal Form . . . 27
Setting up the Component . . . 27
Understanding the FieldTree Type . . . 29
Binding to the Template . . . 30
Working with Schemas . . . 33
Using Separate Schemas . . . 33
Controlling Behavior . . . 34
Debouncing . . . 35
Validating Against Zod and Standard Schema . . . 37

Submitting Forms . . . 38
Logic for Submitting Forms . . . 38
Template for Submitting . . . 40
Further Submit Actions . . . 41
Summary . . . 41

3 State Management with NgRx Signal Store 42
Creating a Signal Store . . . 42
Defining State . . . 44
Providing Resources . . . 46
Providing Computed Signals . . . 47
Providing Methods . . . 48
Setting up Hooks . . . 49
Consuming the Store . . . 49
Inspecting the Store with the Redux DevTools . . . 51
Connecting to the Redux DevTools . . . 51
Disabling DevTools in Production . . . 51
Mutations . . . 54
Creating Mutations . . . 54
Consuming Mutations . . . 57
Using rxMutation . . . 59
Summary . . . 60

4 Signal Queries 61
Content Projection . . . 61
Referencing Parent Components . . . 63
View and Content . . . 65
Interacting with Content . . . 66
Interacting with the View . . . 68
Questioning the Use of viewChild and viewChildren . . . 70
Static Child Components . . . 70
Summary . . . 71

Going Further: Modern Angular 72

Trainings and Consulting 74

Imprint & Legal Notice 75

Preface

Signals are the future of Angular. However, they are more than just a new API: they bring a new paradigm — reactive programming — to the framework. This means we have to rethink the way we design and structure our applications.

In this book, we explore how to use Signals and the Signal Store to build modern, reactive Angular applications. Step by step, we cover the building blocks of reactive Angular — from Signals and Signal-based components to forms, queries, and state management. By the end, you will have a solid foundation for writing maintainable and performant Angular applications the modern way.

Who This Book Is For

This book is written for Angular developers who want to embrace the reactive paradigm and build modern applications with Signals. Some prior Angular knowledge is assumed — you should be familiar with components, services, and the basics of dependency injection. However, you do not need to be a Signals expert: we start with the fundamentals and work our way up to advanced patterns.

The book is also a good fit for architects and lead developers who need to evaluate when and how to adopt Signals and the Signal Store in their projects. If you have already experimented with Signals, it will help you connect the dots and apply them in a structured, scalable way. In short, if you want to bring your Angular skills up to date and build applications that are ready for the years to come, you are in the right place.

Source Code

The complete source code for the examples in this book is available on GitHub[1].

The repository contains the reference implementation used throughout the book and is updated alongside it to reflect current Angular features and practices.

To run the examples, a current LTS version of Node.js, the Angular CLI, Git, and an IDE are required. The author has had good experiences with Visual Studio Code, Cursor AI (for AI-driven development), and WebStorm/IntelliJ. For a quick start, Visual Studio Code is a good choice as it is lightweight and freely available.

[1] https://github.com/angular-architects/flights42

Feedback

Feedback and questions are always welcome. If you have suggestions, found an issue, or would like to share your thoughts, feel free to reach out:

manfred.steyer@angulararchitects.io

If you are interested in applying the concepts from this book in a workshop or real-world setting, feel free to get in touch as well.

About the Author

Manfred Steyer is a trainer, consultant, and architect specializing in enterprise Angular. Together with his team, he helps companies worldwide plan and implement Angular applications that remain maintainable and evolvable in the long term — through workshops, consulting, and hands-on collaboration.

He is the founder of Angular Architects, a company that has conducted over 1,200 workshops and is known for a practical, architecture-first approach to building scalable frontend systems.

Manfred is an Angular GDE, author of several books, and a regular speaker at international conferences, where he shares his experience in designing and implementing modern Angular applications.

Trainings and Consultancy

If you and your team need support or trainings regarding Angular, we are happy to help with workshops and consultancy (on-site or remote). In addition to several other kinds of workshop, we provide the following ones:

- Advanced Angular: Enterprise Solutions and Architecture
- Angular Essentials: Building Blocks and Concepts
- Agentic UI with Angular
- Modern Angular Testing Workshop (Vitest, Playwright, etc.)
- Modern Angular: Best Practices and Patterns

Please find the full list of our offers here[2].

We provide our offer in various forms: **remote** or **on-site**; **public** or as **dedicated company workshops**; in **English** or in **German**.

If you have any questions, reach out to us using office@softwarearchitekt.at.

[2]https://www.angulararchitects.io/en/angular-workshops/



Figure 0-1: Manfred teaching Angular Architecture

1 Reactive Design with Signals

So far, we've just used signals to tell Angular when to update the data bindings in the template. Now, we are going to leverage signals to establish a reactive application design.

That means our code becomes more declarative. Instead of describing how to update dependent values, we just describe how they are derived from other values. The framework takes care of keeping everything in sync. This in turn leads to a more maintainable and less error-prone codebase.

A good analogy is spreadsheets: You define formulas that derive values from other cells. Whenever the source cells change, the dependent cells automatically update. Similarly, with signals, we define relationships between values, and Angular ensures that changes propagate correctly.

In this chapter, we look at the building blocks that enable this reactive design: computed signals, resources, and effects. We also discuss the underlying semantics, such as auto-tracking, untracking, and the glitch-free property. At the end, we'll use all of this together to establish a reactive data flow in our flight search application.

Building Blocks of Reactive Design

For a reactive design with signals, we need to have a way to derive values from existing ones. This can be done synchronously or asynchronously. Also, when the final values don't end up in the template but need to be passed on to other parts of the application, we need a way to react to signal changes in our source code.

This is where computed signals, resources, and effects come into play. The following sections discuss each of these building blocks in detail.

Computed Signals

Computed signals are perhaps the most underrated feature in the signal space. They allow you to define signals that automatically derive their values from other signals. Whenever the dependencies change, the computed signal recalculates its value.

A simple first example of a computed signal in our application is a signal that represents the flight route based on the `from` and `to` **values** stored in our `filter` signal (Listing 1-1).

Listing 1-1

```
// src/app/domains/ticketing/feature-booking/flight-search/flight-search.ts

import { DatePipe, JsonPipe } from '@angular/common';
import { Component, computed, signal } from '@angular/core';
import { FormField, form } from '@angular/forms/signals';

[...]

@Component([...])
export class FlightSearch {
  protected readonly filter = signal({
    from: 'Hamburg',
    to: 'Graz',
  });

  protected readonly filterForm = form(this.filter);

  // Computed signal automatically updates when filter changes
  protected readonly flightRoute = computed(
    () => `${this.filter().from} to ${this.filter().to}`,
  );

  // ... rest of component
}
```

The result of `computed` is a read-only signal that can be used in the template like any other signal:

```
<h2>Search Route: {{ flightRoute() }}</h2>
```

Whenever the user changes the search criteria via the form fields, the `filter` signal changes. As a consequence, the `flightRoute` signal automatically updates, and Angular updates the displayed route in the template.

> **Remarks on Updating Computed Signals**
>
> Actually, computed signals are updated lazily. This means they are recalculated only when the application is interested in them, e.g., when they're read in the template or when the code calls the getter.

If a building block watches a Signal for changes, we say it *tracks* the signal. However, there are scenarios where you might want to read a signal within `computed` without tracking it. For such cases, Angular provides the `untracked()` function.

The following listing first extracts `from` and `to` from the filter and then uses `untracked` with the latter one (Listing 1-2).

Listing 1-2

```
// src/app/domains/ticketing/feature-booking/flight-search/flight-search.ts
import { Component, signal, computed, untracked } from '@angular/core';

@Component({
  /* ... */
})
export class FlightSearch {

  protected readonly filter = signal({
    from: 'Hamburg',
    to: 'Graz',
  });

  [...]

  protected readonly from = computed(() => this.filter().from);
  protected readonly to = computed(() => this.filter().to);

  // Only tracks 'from', not 'to'
  protected readonly flightRoute = computed(() => {
    const origin = this.from(); // tracked
    const destination = untracked(() => this.to()); // NOT tracked
    return `${origin} → ${destination}`;
  });

  // ... rest of component
}
```

Here, `flightRoute` only updates when `from` changes, not when `to` changes. This is useful when you want fine-grained control over reactivity.

Resources

While computed signals are great for deriving synchronous values, many applications need to deal with asynchronous data, such as fetching data from a backend. This is where resources come into play. In this section, we discuss three resource implementations Angular provides: `httpResource`, `rxResource`, and the Promise-based `resource`.

httpResource

Resources asynchronously project input signals to output signals. As in most cases, this asynchronous projection is an HTTP request, Angular provides the already discussed `httpResource` as a convenient high-level API (Listing 1-3).

Listing 1-3

```
import { httpResource } from '@angular/common/http';

import { Flight, FlightZodSchema } from '../../data/flight';

[...]

@Component([...])
export class FlightSearch {

  protected readonly filter = signal({
    from: 'Graz',
    to: 'Hamburg',
  });

  [...]

  protected readonly flightsResource = httpResource<Flight[]>(
    () => {
      const filter = this.filter();
      if (!filter.from || !filter.to) {
        return undefined;
      }

      return {
        url: `https://demo.angulararchitects.io/api/flight`,
        params: {
          from: filter.from,
          to: filter.to,
        },
      };
    },
    {
      defaultValue: [],
      parse: (raw) => FlightZodSchema.array().parse(raw),
    },
  );

  protected readonly flights = this.flightsResource.value;
  protected readonly isLoading = this.flightsResource.isLoading;
  protected readonly error = this.flightsResource.error;

  [...]
}
```

The first argument is a function that returns the HTTP request to perform. Internally, it's converted into a computed signal. Hence, it automatically tracks the `filter` signal read inside the function. Whenever this computed signal changes, the resource triggers a new HTTP request. Also, it triggers the request initially when created.

To deactivate the resource, the request function returns `undefined`. In our case, this happens when either `filter.from` or `filter.to` is empty.

The second argument is an options object. It allows us to define a default value that avoids the need to handle `undefined` values before the first request completes.

Besides this, this options object provides a feature we have not discussed so far: The `parse` function. It allows for transforming and validating the raw response before it's set as the resource's value.

In our example, `parse` delegates to a Zod schema[1] to ensure we get valid data from the backend. Using schemas also helps to revive data types such as dates that are lost during JSON serialization. For instance, if the Zod schema defines a date property, it automatically converts the respective string value from the JSON response into a Date object.

Under the hood, `httpResource` uses the `HttpClient` for performing the HTTP request.

rxResource

Another resource implementation Angular provides is the `rxResource`. It allows specifying a stream loader (property `stream`) returning an Observable that can emit several values over time. This Observable is used to populate the status and result signals such as `isLoading`, `error`, and `value` (Listing 1-4).

Listing 1-4

```
import { rxResource } from '@angular/core/rxjs-interop';

[...]

@Component([...])
export class FlightSearch {

  private readonly http = inject(HttpClient);

  protected readonly filter = signal({
    from: 'Graz',
    to: 'Hamburg',
  });

  [...]

  protected readonly flightsResource = rxResource({
    params: () => ({
      ...this.filter(),
```

[1] https://zod.dev/

```
    }),
    stream: (loaderParams) => {
      const c = loaderParams.params;
      return this.find(c.from, c.to);
    },
    defaultValue: [],
  });

  protected readonly flights = this.flightsResource.value;
  protected readonly isLoading = this.flightsResource.isLoading;
  protected readonly error = this.flightsResource.error;

  [...]
}
```

Similar to the request in the `httpResource`, the `params` property is converted into a computed signal under the hood. Whenever one of the signals read in there changes, the resource's stream loader is retriggered. The resource also triggers the loader when it is created.

If you want to avoid the resource from triggering the loader, you can return undefined from the `params` function. The `defaultValue` parameter avoids having to handle `undefined` values before the loader returns a result.

When the resource calls the stream loader, it passes an object containing the current parameters as ordinary data types, not signals. The shown example delegates to the method `find` that uses the `HttpClient` for fetching data as an Observable (Listing 1-5).

Listing 1-5

```
find(from: string, to: string, urgent = false): Observable<Flight[]> {
  const url = `https://demo.angulararchitects.io/api/flight`;

  const headers = {
    Accept: 'application/json',
  };

  const params = { from, to, urgent };

  return this.http
    .get<Flight[]>(url, { headers, params });
}
```

This approach is especially useful when you already have a service that uses Observables, or when you need the power of RxJS to compose complex data streams. From the consumer's perspective, it is just a resource like the `httpResource`; internally, it delegates to an Observable stream.

> **Remarks: RxJS Interop**
>
> Perhaps you've recognized that the `rxResource` is imported from the `@angular/core/rxjs-interop` package. This package contains utilities for integrating RxJS with Angular Signals. There are some further interesting methods that allow us to bridge between both worlds. Examples are `toSignal` for converting an Observable to a Signal and `toObservable` for the other way around.

resource (Promise-based)

Both the `httpResource` and the `rxResource` are abstractions of the underlying Promise-based resource implementation just called `resource`. In practice, you will rarely use it directly in applications. For the sake of completeness, here is what it looks like (Listing 1-6).

Listing 1-6

```
import { Component, signal, resource } from '@angular/core';

[...]

@Component([...])
export class FlightSearch {

  protected readonly filter = signal({
    from: 'Graz',
    to: 'Hamburg',
  });

  [...]

  protected readonly flightsResource = resource({
    params: () => ({
      from: this.filter().from,
      to: this.filter().to,
    }),
    loader: (loaderParams) => {
      const c = loaderParams.params;
      return this.findPromise(c.from, c.to);
    },
    defaultValue: [],
  });

  protected readonly flights = this.flightsResource.value;
  protected readonly isLoading = this.flightsResource.isLoading;
  protected readonly error = this.flightsResource.error;
```

```
  [...]
}
```

The main difference from the `rxResource` is that it uses a `loader` function that returns a Promise instead of a `stream` function that returns an Observable. Also, here, the API and semantics from the caller's perspective are the same.

Like all resources, this base implementation also handles race conditions: When multiple requests are triggered in quick succession, only the result of the latest request is used, discarding any earlier responses that arrive later. This prevents outdated information from being displayed in the UI. For this, the `httpResource` and `rxResource` can cancel ongoing requests that are no longer needed. Since this does not directly work with Promises, `resource` simply ignores the results of outdated requests.

To enable cancellation, the called function must respect an `AbortSignal` passed via the loader parameters. Other than the name suggests, an `AbortSignal` has nothing to do with Angular's signal implementation. It's a browser API for canceling asynchronous operations. Technically, it provides an event that notifies when the operation should be aborted (Listing 1-7).

Listing 1-7

```
import { Component, resource, signal } from '@angular/core';

import { firstValueFrom } from 'rxjs';

[...]

@Component([...])
export class FlightSearch {

  protected readonly filter = signal({
    from: 'Graz',
    to: 'Hamburg',
  });

  protected readonly flightsResource = resource({
    params: () => ({
      from: this.filter().from,
      to: this.filter().to,
    }),
    loader: (loaderParams) => {
      const c = loaderParams.params;
      const abortSignal = loaderParams.abortSignal;
      return this.findPromise(c.from, c.to, abortSignal);
    },
    defaultValue: [],
  });
```

```
  [...]
}
```

As the `HttpClient` always returns an Observable, our implementation of `findPromise` needs to convert it to a Promise. For this, we can use the `firstValueFrom` function from RxJS.

Listing 1-8

```
import { firstValueFrom, Subject, takeUntil } from 'rxjs';

[...]

findPromise(
  from: string,
  to: string,
  abortSignal?: AbortSignal,
): Promise<Flight[]> {
  const aborted = new Subject<void>();

  abortSignal?.addEventListener('abort', () => {
    aborted.next();
  });

  const flightsObservable = this.find(from, to).pipe(takeUntil(aborted));
  return firstValueFrom(flightsObservable);
}
```

When the passed `AbortSignal` emits an `abort` event, we need to unsubscribe from the Observable to cancel the ongoing HTTP request. In RxJS, this can be done via the `takeUntil` operator that completes the Observable when another Observable (or Subject) emits a value. Here, the `aborted` Subject serves this purpose.

Please note that `findPromise` mainly exists for demonstration purposes. In a real application, you would very likely stick with the Observable you already have and use the `rxResource` instead.

Effects

Similar to computed signals, an effect is a piece of code that automatically runs whenever a signal it reads changes. However, effects don't return a value but execute a side effect, such as logging, updating the DOM, or painting on a canvas.

Here, we add an effect that logs the search criteria whenever they change (Listing 1-9).

Listing 1-9

```ts
// src/app/domains/ticketing/feature-booking/flight-search/flight-search.ts
import { Component, signal, effect } from '@angular/core';

@Component({
  /* ... */
})
export class FlightSearch {
  protected readonly filter = signal({
    from: 'Hamburg',
    to: 'Graz',
  });

  constructor() {
    // Effect runs whenever the filter changes
    effect(() => {
      const filter = this.filter();
      console.log('From:', filter.from);
      console.log('To:', filter.to);
    });
  }

  // ... rest of component
}
```

> **Remarks: Effects and Injection Context**
>
> Effects need to be called in a so-called injection context. To cut a long story short: Your component's constructor and field initializers are always executed in such a context.
>
> In general, the constructor is the right place for setting up effects. Think about installing cables in a new house: You would typically install all cables when the house is built (constructor), and after that, you just use it for sending data (and power) through it.

Effects should be used sparingly, as they can lead to less maintainable code. Just imagine a chain of effects triggering each other – debugging such a scenario would be a nightmare. For this reason, computed signals should be preferred whenever possible.

As a rule of thumb, mainly use effects for rendering that cannot be done via data binding, such as painting on a canvas, displaying a toast message, or delegating to third-party libraries that are not aware of signals. The term *rendering* is used here in a broader sense to mean "going the last mile" to the data sink, which is usually the UI but can also be other targets such as the browser console (logging) or local storage in the browser.

A good use of an effect in our application is to display a toast message whenever an error occurs, since this cannot be done via data binding. For this, we use Angular Material's `MatSnackBar` (Listing 1-10).

Listing 1-10

```
import {
  effect,
} from '@angular/core';
import { MatSnackBar } from '@angular/material/snack-bar';

[...]

@Component( [...])
export class FlightSearch {

  private readonly snackBar = inject(MatSnackBar);

  [...]

  constructor() {
    this.showError();
  }

  [...]

  private showError() {
    effect(() => {
      const error = this.error();
      if (error || this.filter().to === 'error') {
        const message = 'Error loading flights: ' + error;
        this.snackBar.open(message, 'OK');
      }
    });
  }
}
```

If your effect needs to manipulate the DOM or read details from the DOM, you need to make sure that the effect runs after Angular is done with rendering. For exactly this case, Angular provides the `afterRenderEffect` function:

```
afterRenderEffect(() => {
  // DOM manipulation here
});
```

Examples include measuring element sizes, scrolling to a specific position, or integrating with third-party libraries that manipulate the DOM, such as charting libraries.

If you want to execute a specific DOM-based logic after the next rendering cycle, regardless of whether a read signal was changed or not, you can use `afterNextRender` (Listing 1-11).

Listing 1-11

```
afterNextRender(() => {
  const filter = this.filter();
  console.log('After Next Render: from', filter.from);
  console.log('After Next Render: to', filter.to);
});
```

If you want to execute logic after every rendering cycle, independently of signal changes, you can use `afterEveryRender`.

Listing 1-12

```
afterEveryRender(() => {
  const filter = this.filter();
  console.log('After Every Render: from', filter.from);
  console.log('After Every Render: to', filter.to);
});
```

Signal Semantics in Angular

Signals are easy at first glance. You create, read, and update them. However, to use signals effectively in a reactive design, it's important to understand some of the underlying semantics. This section discusses them.

Signals in the Component Lifecycle

When setting up an effect, it runs once and then, whenever a signal changes, it is re-executed. However, when the effect is created within a component, e.g., in the constructor, Angular defers its execution until the component is fully initialized. That means, when the effect runs, it can safely read input properties (Listing 1-13).

Listing 1-13

```
@Component( [...] )
export class FlightCard {
  readonly item = input.required<Flight>();

  [...]

  constructor() {
    // console.log('Item:', this.item());
    //  Error: item not initialized yet

    effect(() => {
      console.log('Item:', this.item());
    });
  }

  [...]
}
```

Accessing inputs directly in the constructor would cause an error because the inputs are not yet initialized. The constructor is called when the component is instantiated, and only after Angular can perform the initial data binding.

For the same reason, it is safe to read input properties inside computed signals (Listing 1-14).

Listing 1-14

```
@Component( [...] )
export class FlightCard {
  readonly item = input.required<Flight>();

  protected readonly flightRoute = computed(
    () => `${this.item().from} to ${this.item().to}`
  );
  [...]
}
```

Auto-Tracking and the Reactive Context

Angular automatically tracks all signals read inside a so-called reactive context. From an application developer's viewpoint, Angular creates a reactive context for only two building blocks: templates and effects. Signals read inside computed signals are tracked too when read inside templates or effects. However, in this case, the reactive context of the template or effect is used.

Angular will even track a Signal that is read inside a method or function called in a reactive context (Listing 1-15).

Listing 1-15

```
// src/app/domains/ticketing/feature-booking/flight-search/flight-search.ts
import { Component, signal, effect } from '@angular/core';

@Component({
  /* ... */
})
export class FlightSearch {
  protected readonly filter = signal({
    from: 'Hamburg',
    to: 'Graz',
  });

  constructor() {
    // Effect automatically tracks all signals read in logCriteria
    effect(() => {
      this.logCriteria();
    });
  }

  private logCriteria(): void {
    // The filter signal is automatically tracked
    const filter = this.filter();
    console.log('Criteria:', filter.from, '→', filter.to);
  }

  // ... rest of component
}
```

Here, by just looking at the effect, it's not obvious that it tracks `filter`. However, because `logCriteria` is called inside the effect, it reruns whenever the filter changes.

Above, I mentioned that effects are often used for rendering that cannot be done via data binding. In such a case, this behavior might be exactly what you want. Let's say you are using an effect to render a diagram on a canvas. When one of the underlying values changes, you want to re-render the diagram.

However, there are also situations where you don't want this behavior. For instance, consider the following effect that triggers some business logic (Listing 1-16).

Listing 1-16

```
effect(() => {
  const criteria = this.criteria();
  this.businessService.executeLogic(criteria);
  //  Don't trigger business logic in effects
});
```

Here, at first glance, we get the impression that the effect only reruns when the `criteria` property changes. However, if `executeLogic` reads other signals internally, those will be tracked too. This leads to unexpected reruns of the effect. Let's assume `executeLogic` reads the signals `isLoading` and `userId` before deleting some records. Every time the `isLoading` flag or the `userId` changes in this or another part of the application, the effect will rerun and delete further records.

For this reason, you usually don't call business logic in effects. Another reason for preventing effects from triggering business logic is that, unlike the Resource API, an effect does not handle race conditions. For instance, overlapping calls might overwrite each other's results.

Discussing Explicit Effects

The `untracked` function can also be used inside effects to prevent tracking. This allows to implement a so-called explicit effect. The idea is to only track signals that are explicitly marked for tracking (Listing 1-17).

Listing 1-17

```
//  Only triggers when criteria changes
effect(() => {
  const criteria = this.criteria();
  untracked(() => {
    this.businessService.executeLogic(criteria);
  });
});
```

Because the business logic is wrapped in `untracked`, changes to signals read inside `executeLogic` will not trigger the effect. Or in other words: Only changes in `criteria` will lead to a rerun.

This pattern is highly controversial in the community. While it solves the discussed problem, it also makes the code less transparent. It's not in the spirit of reactive programming, where values are derived from one another. Also, it might lead to hard-to-debug chain reactions and cycles, and it doesn't address race conditions.

Untracking

To avoid memory leaks, Angular stops tracking signals when the underlying building block, e.g., the component, is destroyed. Also, they are untracked when not read (anymore) during a run (Listing 1-18).

Listing 1-18

```
effect(() => {
  if (isDelayed()) {
    console.log(delay());
  }
});
```

In this example, `delay` is only tracked when `isDelayed` is `true`. If `isDelayed` becomes `false`, `delay` is no longer read and is hence untracked. That means that changes in `delay` will no longer trigger re-execution of the effect.

In most situations, this behavior is completely fine. However, if you want those changes to a Signal `delay` always trigger an effect, you need to read it outside of the condition. Usually, this is done at the beginning (Listing 1-19).

Listing 1-19

```
effect(() => {
  const isDelayedValue = isDelayed();
  const delayValue = delay();
  if (isDelayedValue) {
    console.log(delayValue);
  }
});
```

Here, I'm even reading `isDelayed` outside of the condition. While not necessary, this "pattern" makes the code more consistent and assures that I'm on the safe side.

While the example here is effect-based, the same applies to computed and signals used in templates.

Glitch-Free Property

Angular's signals are **glitch-free**, meaning that a reactive consumer, such as a template or an effect, never sees inconsistent intermediate states. When you change multiple signals, the consuming reactive context only runs once with the final values. Consider this example (Listing 1-20).

Listing 1-20

```
protected readonly filter = signal({
  from: 'Hamburg',
  to: 'Graz',
});

constructor() {
  effect(() => {
    const filter = this.filter();
    console.log('from:', filter.from);
    console.log('to:', filter.to);
  });

  setTimeout(() => {
    this.filter.update((filter) => ({ ...filter, from: 'Paris' }));
    this.filter.update((filter) => ({ ...filter, from: 'Frankfurt' }));
    this.filter.update((filter) => ({ ...filter, from: 'New York' }));

    this.filter.update((filter) => ({ ...filter, to: 'Berlin' }));
    this.filter.update((filter) => ({ ...filter, to: 'Zurich' }));
    this.filter.update((filter) => ({ ...filter, to: 'London' }));
  }, 2000);
}
```

After 2 seconds, the effect runs only once and prints out `New York` and `London`. This behavior prevents inconsistent states but also unnecessary rendering. However, this also emphasizes that Signals are not suited for representing events or time-based data streams, as messages that are quickly followed by others might get lost. For such scenarios, RxJS and Observables are a better fit.

Equality and Immutability

When updating a signal, Angular checks whether the value has indeed changed to avoid unnecessary updates of dependent signals and re-renderings. For instance, in the following case, nothing needs to happen (Listing 1-21).

Listing 1-21

```
const count = signal(0);

[...]

count.set(0);
count.set(0);
count.set(0);
```

By default, Angular uses strict equality (===) for this check. For primitive types like strings and numbers, this works well. However, for objects and arrays, === checks the object reference, not the content. Therefore, you need to ensure that you create new object instances and copy over all parts that have not changed. This can be done easily with the spread operator (Listing 1-22).

Listing 1-22

```
flight.update((flight) => ({
  ...flight,
  date: newDate,
  delayed: true,
}));
```

Here, `...flight` creates a shallow copy of the original flight object. Then, the example overrides the `date` and `delayed` properties with new values. The resulting object is a new instance, so Angular detects the change and updates dependent signals and the UI accordingly.

This is a common practice in functional and reactive programming known as immutability. Immutable data structures never change their content; instead, new versions are created when changes are needed.

Remarks: Immutability and OnPush

Angular's optimized change detection strategy, `OnPush`, which we have been using from the beginning, also relies on immutability. Let's think about our `FlightSearch` component. As `flights` is a Signal, Angular can find out when the `flights` array has been changed, but it also needs to know which `flight` in the array has changed to only update the respective flight card (Listing 1-23).

Listing 1-23

```
<!-- .../ticketing/feature-booking/flight-search/flight-search.html -->

@for (flight of flights(); track flight.id) {
  <div class="col-xs-12 col-sm-6 col-md-4 col-lg-3">
    <app-flight-card
      [item]="flight"
      [selected]="basket()[flight.id]"
      (selectedChange)="updateBasket(flight.id, $event)"
    >
      <button [routerLink]="['../flight-edit', flight.id, { showDetails: true }]">
        Edit
      </button>
    </app-flight-card>
  </div>
}
```

> For this task, Angular is also performing a strict equality check (`===`) between the old and new flight object. If there were no changes, Angular would not need to update the respective `FlightCard` component. Otherwise, it takes a closer look at the changed flight card to update its bindings.
>
> So, to cut a long story short: Always create new object instances when updating bound objects or arrays. This way, Angular can efficiently detect changes and update the UI accordingly.

For special cases, you can provide a custom equality function. For instance, the `signal` function accepts an optional second parameter for this purpose. This option is rarely needed in practice, especially in application code, and might create more confusion than benefits in most cases.

Establishing a Reactive Flow

Now that we have discussed the building blocks and semantics of reactive design with Signals, let's put everything together to establish a complete reactive data flow in our flight search application. This results in a declarative design in which we describe how information is derived from other information. Hence, we don't need to worry about manually updating dependent values at specific events.

Thinking in Terms of the Signal Graph

When working with Signals, Angular maintains a so-called signal graph in the background. This is a data structure that represents how signals, computed signals, and consumers, such as effects and templates, depend on each other. In other words, it shows how data flows through the application. When we think in terms of this graph, establishing a reactive flow feels quite natural.

For instance, let's assume that besides flights, we also have some information on delays. A signal graph that brings both together could look like in Figure 1-1.

Here, the flow starts with `filter` that is asynchronously projected to the loaded `flights` via the `flightsResource`. The resulting `flights` signal is then combined with the `delayInMin` signal to create a new computed signal `flightsWithDelays` that contains the final data displayed in the template.

For the sake of simplicity, let's assume that the `delayInMin` signal just represents a client-side value that only affects the first loaded flight. You might see it as a simulation exercise by the pilots' union.

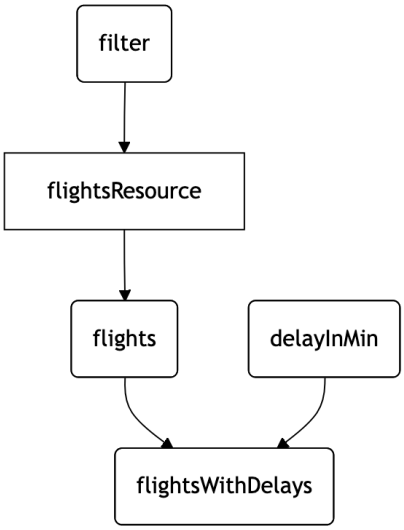


Figure 1-1: Reactive Flow with Signals

Implementing the Reactive Flow

To implement the discussed reactive flow, we need to add the `delayInMin` signal and the computed signal `flightsWithDelays` to our `FlightSearch` component (Listing 1-24).

Listing 1-24

```
// src/app/domains/ticketing/feature-booking/flight-search/flight-search.ts

import { JsonPipe } from '@angular/common';
import {
  ChangeDetectionStrategy,
  Component,
  computed,
  inject,
  signal,
} from '@angular/core';
import { httpResource } from '@angular/common/http';
import { FormField, form } from '@angular/forms/signals';

import { Flight } from '../../data/flight';
import { FlightCard } from '../../ui/flight-card/flight-card';

@Component({
  selector: 'app-flight-search',
  imports: [FormField, FlightCard, JsonPipe],
  templateUrl: './flight-search.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
```

```
})
export class FlightSearch {
  protected readonly filter = signal({
    from: 'Graz',
    to: 'Hamburg',
  });

  protected readonly filterForm = form(this.filter);

  protected readonly flightsResource = httpResource<Flight[]>( [...] );

  protected readonly flights = this.flightsResource.value;
  protected readonly error = this.flightsResource.error;
  protected readonly isLoading = this.flightsResource.isLoading;

  protected readonly delayInMin = signal(0);
  protected readonly flightsWithDelays = computed(() =>
    toFlightsWithDelays(this.flights(), this.delayInMin()),
  );

  protected readonly basket = signal<Record<number, boolean>>({});

  protected search(): void {
    this.flightsResource.reload();
  }

  protected updateBasket(flightId: number, selected: boolean): void {
    this.basket.update((basket) => ({
      ...basket,
      [flightId]: selected,
    }));
  }

  protected delay(): void {
    this.delayInMin.update((delayInMin) => delayInMin + 15);
  }
}
```

The computed signal `flightsWithDelays` delegates to a pure helper function `toFlightsWithDelays` that is not part of the class and combines the flights with the delay information (Listing 1-25).

Listing 1-25

```
function toFlightsWithDelays(flights: Flight[], delay: number): Flight[] {
  if (flights.length === 0) {
    return [];
  }

  const ONE_MINUTE = 1000 * 60;
  const oldFlights = flights;
  const oldFlight = oldFlights[0];
  const oldDate = new Date(oldFlight.date);
  const newDate = new Date(oldDate.getTime() + delay * ONE_MINUTE);
  const newFlight = { ...oldFlight, date: newDate.toISOString() };

  return [newFlight, ...flights.slice(1)];
}
```

As discussed before, the flight arrays and the flights are treated as immutable objects. Therefore, the helper function creates a new flight object and copies over the unchanged properties. The same applies to the flight array.

> I prefer pure helper functions placed outside of the class over methods for such cases. On the one hand, they are easier to reason about as they cannot access any instance state. On the other hand, delegating to pure functions that don't take signals as arguments demands us to read the signals before calling the function, e.g., directly inside `computed`. Hence, we see at first glance which signals influence the computed value.

The example discussed here shows another interesting aspect: the `delay` method simply updates the `delayInMin` signal. In a classic implementation, it would update the `flight` array directly. That means we needed to remember when and where flights need to be updated, and seeing the big picture would be hard. Also, it would be difficult to find out how a specific state came to be.

To try out our modifications, our template needs a delay button and must bind to `flightsWithDelays` instead of `flights` (Listing 1-26).

Listing 1-26

```
<button type="button" (click)="delay()" [disabled]="flights().length === 0">Delay</button>

[...]

@for (flight of flightsWithDelays(); track flight.id) {
  <div class="col-xs-12 col-sm-6 col-md-4 col-lg-3">
    <app-flight-card [...]>
      [...]
    </app-flight-card>
  </div>
}
```

Summary

In this chapter, we explored how to leverage Signals to establish a reactive design in Angular applications. We discussed the building blocks, such as computed signals, resources, and effects, that enable this design. We also delved into the underlying semantics, such as auto-tracking, untracking, and the glitch-free property.

By thinking in terms of the signal graph, we designed a declarative and maintainable application structure that automatically keeps everything in sync. This approach not only simplifies our code but also enhances its reliability and scalability.

2 Signal Forms

Signal Forms bridge a crucial gap between Angular's Signal-based reactivity and user interaction. They provide a powerful and flexible way to manage form state, validation, and submission logic in Angular applications.

In this chapter, I'll go into the details of this new library. In addition to the basics, I'll discuss more advanced aspects such as the many types of custom validators, conditional validation, subforms, and custom controls.

For this, we are going to extend a `FlightEdit` component (Figure 2-1).

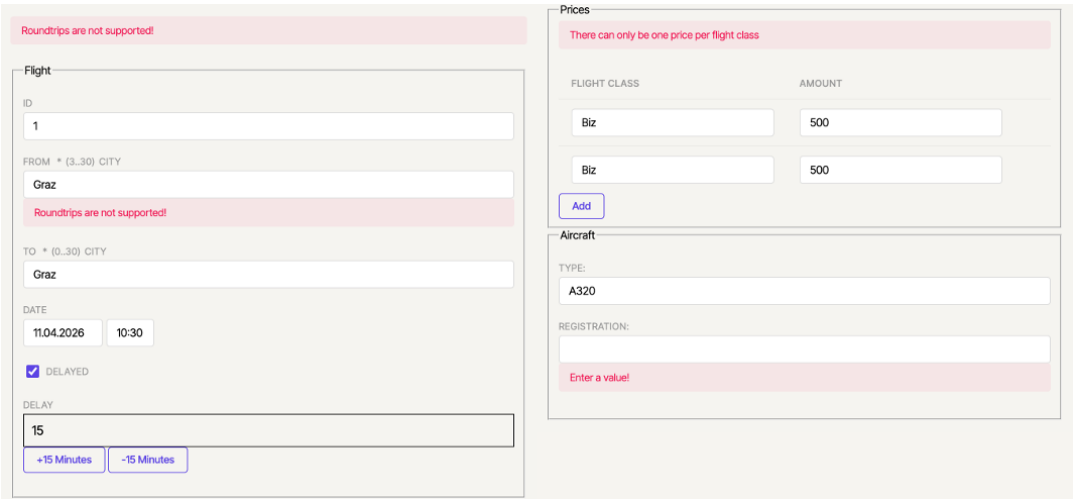


Figure 2-1: The FlightEdit component covers most Signal Forms features

A First Signal Form

To get started, we create a first form for the flight itself. In later sections, we will extend it to include nested forms for the connected airplane and the prices.

Setting up the Component

Our implementation receives the flight to be displayed from a service named `FlightDetailStore`. To avoid repetition, we won't go into its internals here. If you want to implement the concepts shown step by step, you can find a simplified

version of this store for this chapter in the file `simple-flight-detail-store.ts`. In that case, also take a look at the `FlightClient`, which provides some additional methods used by this store.

Since a store is responsible for data consistency, it only publishes read-only data. On the other hand, we want to modify the data using the form: A two-way binding should transfer the user input directly to the bound `flight` and vice versa. Hence, we need a local working copy that can be represented by a linked signal (Listing 2-1).

Listing 2-1

```
// src/app/domains/ticketing/feature-booking/flight-edit/flight-edit.ts

[...]

import { linkedSignal } from '@angular/core';
import { form, minLength, required } from '@angular/forms/signals';
[...]

@Component([...])
export class FlightEdit {
  private readonly store = inject(FlightDetailStore);

  protected readonly flight = linkedSignal(() =>
    normalizeFlight(this.store.flightValue()),
  );

  // Set up the Signal Form with validation rules
  protected readonly flightForm = form(this.flight, (path) => {
    required(path.from);
    required(path.to);
    required(path.date);

    minLength(path.from, 3);
  });

  [...]
}
```

The helper function `normalizeFlight` converts the flight date into the right format to be used with an `<input type="datetime-local">`. This is an ISO string without a timezone designator at the end, e.g., `2030-12-24T17:30:00.000`.

Listing 2-2

```
function normalizeFlight(flight: Flight): Flight {
  const localDate = flight.date.substring(0, 16);
  return {
    ...flight,
    date: localDate,
  };
}
```

The `form` function from Angular's Signal Forms package gets the linked signal representing the flight to edit.

The second parameter that is passed to `form` is a schema with validation rules. Angular provides the `required` and `minLength` rules out of the box. Further simple validators that ship with Signal Forms include `maxLength`, `min`, `max`, and `pattern`. The latter checks against a regular expression.

The `path` parameter of type `SchemaPathTree` is used to reference the individual properties we want to validate, e.g., `from` or `to`.

As a result, `form` returns a `FieldTree` that allows binding the individual properties of the flight to form controls in the template. We'll discuss this important data structure in the next section.

Understanding the FieldTree Type

You can think of a `FieldTree` as a deeply nested Signal. Each property in the data structure is represented by a Signal with form state, e.g., `value`, `dirty`, `invalid`. Because these properties are Signal-based, we can bind them to form controls.

For instance, let's imagine the `flight` we pass to the function `form` looks like this (Listing 2-3).

Listing 2-3

```
{
  id: 1,
  from: 'Graz',
  to: 'Hamburg',
  date: '2030-12-24T17:30',
  delayed: false,
  delay: 0,
  aircraft: {
    type: 'T0815',
    registration: 'R4711'
  },
  prices: [
```

```
    { flightClass: 'economy', amount: 299 },
    { flightClass: 'business', amount: 599 },
  ]
}
```

In this case, the `FieldTree` returned by the `form` provides Signals for each property (Listing 2-4).

Listing 2-4

```
const date = this.flightForm.date().value();
const isDateDirty = this.flightForm.date().dirty();
const isDateInvalid = this.flightForm.date().invalid();
const dateErrors = this.flightForm.date().errors();
```

Here, the `date` itself is a signal, and `value`, `dirty`, `invalid`, and `errors` are as well. The property `dirty` is `true` when the user modified the value, and `invalid` is `true` when there was a validation error.

The `errors` property contains an array with all detected validation errors. For instance, the schema setup in the previous section requires `from` to be set. If this is not the case, this violation is reported by an object in the `errors` array. To get a better feeling for it, we'll display this array with the `JsonPipe`.

As the `FieldTree` is nested, we can also access the properties at deeper levels, such as the airplane's type or the prices (Listing 2-5).

Listing 2-5

```
const aircraftType = this.flightForm.aircraft.type().value();
const isAircraftTypeDirty = this.flightForm.aircraft.type().dirty();
[...]

const firstPriceAmount = this.flightForm.prices[0].amount().value();
const isFirstPriceAmountDirty = this.flightForm.prices[0].amount().dirty();
[...]
```

In our example, that means we can bind the properties of the flight itself, as well as those of the connected airplane and the connected prices. The next section is already binding some properties to the template.

Binding to the Template

For binding parts of the Signal Form to the template, we need to import the `FormField` directive. For easily displaying validation errors during our first steps, we also import the `JsonPipe` (Listing 2-6).

Listing 2-6

```
// src/app/domains/ticketing/feature-booking/flight-edit/flight-edit.ts

[...]

import { JsonPipe } from '@angular/common';

import {
  form,
  FormField,
  minLength,
  required,
} from '@angular/forms/signals';

@Component({
  selector: 'app-flight-edit',
  imports: [
    [...]

    // Import FormField directive
    FormField,

    // Add JsonPipe
    JsonPipe,
  ],
  templateUrl: './flight-edit.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class FlightEdit {
  [...]
}
```

This directive can be used to bind individual fields of our `FieldTree` to form controls such as `input`, `select`, and `textarea` (Listing 2-7).

Listing 2-7

```
<!-- .../ticketing/feature-booking/flight-edit/flight-edit.html -->

<fieldset>
  <legend>Flight</legend>

  <div class="form-group">
    <label for="flight-id">ID</label>
    <input
      class="form-control"
      [formField]="flightForm.id"
      type="number"
      id="flight-id"
    />
  </div>

  <div class="form-group">
    <label for="flight-from">From</label>
    <input
      class="form-control"
      [formField]="flightForm.from"
      id="flight-from"
    />
    @if (flightForm.from().invalid()) {
      <div>{{ flightForm.from().errors() | json }}</div>
    }
  </div>
  […]
</fieldset>
```

Please note that when binding a number, you need to use an `<input type="number" ...>` because Signal Forms also respects HTML semantics when ensuring type safety.

The `errors` property is an array containing all validation errors detected for the field, which the component outputs for illustrative purposes. The following screenshot shows this array in a case where the `minLength` validator was triggered (Figure 2-2).



Figure 2-2: Validation Errors

However, the `minLength` validator isn't triggered for empty fields. This common approach supports optional fields. However, if the field isn't optional, a `required` validator handles the empty field.

Working with Schemas

Besides validation rules, the schema passed to the *form* function also defines other aspects of the form's behavior. For instance, it could specify that changes are debounced or that some fields are read-only under certain conditions. In this section, I'll discuss these aspects in more detail.

Using Separate Schemas

So far, the schema has been defined directly in the call to the `form` function. However, for larger schemas, this approach quickly becomes unwieldy. To make the individual components clearer, the schema can be refactored to a constant in a separate file. Since the schema defines general rules, it can be placed in the `data` folder.

Listing 2-8

```
// src/app/domains/ticketing/data/flight-schema.ts

import { required, minLength, schema } from '@angular/forms/signals';

import { Flight } from './flight';

export const flightSchema = schema<Flight>((path) => {
  required(path.from);
  required(path.to);
  required(path.date);

  minLength(path.from, 3);
});
```

The call to the function `form` can then refer to it (Listing 2-9).

Listing 2-9

```
// src/app/domains/ticketing/feature-booking/flight-edit/flight-edit.ts

import { flightSchema } from '../../data/flight-schema';
[...]

protected readonly flightForm = form(this.flight, flightSchema);
[...]
```

Schemas can also reference each other. This is useful, for example, if a form adds some specific validation rules to a schema with general ones (Listing 2-10).

Listing 2-10

```
import {
  apply,
  minLength,
  required,
  schema,
} from '@angular/forms/signals';

import { flightSchema } from '../../data/flight-schema';

export const flightFormSchema = schema<Flight>((path) => {
  apply(path, flightSchema);
  required(path.id);
});
```

The shown `flightFormSchema` uses `apply` to include all rules defined in `flightSchema`. In addition, it requires the flight's ID to be set.

Another case where schemas refer to other schemas is conditional validation rules. In the next section, we will discuss this.

Controlling Behavior

Schemas not only define validation rules but also further behavior. For instance, the schema can also specify under which circumstances the `formField` directive should disable a field (Listing 2-11).

Listing 2-11

```
// src/app/domains/ticketing/data/flight-schema.ts

import { disabled } from '@angular/forms/signals';

[...]
protected readonly filterForm = form(this.filter, (schema) => {
  [...]
  disabled(path.delay, (ctx) => !ctx.valueOf(path.delayed));
}
[...]
```

Instead of a boolean, you can also return a reason for the disabling (Listing 2-12).

Listing 2-12

```
// src/app/domains/ticketing/data/flight-schema.ts
protected readonly filterForm = form(this.filter, (schema) => {
  [...]
  disabled(path.delay, (ctx) =>
    ctx.valueOf(path.delayed) ? false : 'not delayed',
  );
}
```

All the reasons why a field is currently disabled are found in its `disabledReasons` Signal:

```
@for (reason of flightForm.delay().disabledReasons(); track $index) {
  <p>Disabled because {{ reason.message }}</p>
}
```

The functions `hidden` and `readonly` can be used to hide or make fields read-only in the same way. However, they always return boolean values and don't support reasons.

While Signal Forms automatically disables writing to bound fields, e.g., input elements, marked as read-only, hiding fields is left to the application. This is because, typically, hiding a field also requires hiding its label and perhaps other UI elements. Hence, hidden is only a hint that your template can use (Listing 2-13).

Listing 2-13

```
@if (!flightForm.delay().hidden()) {
  <div class="form-group">
    <label for="flight-delay">Delay (min)</label>
    <input
      class="form-control"
      [formField]="flightForm.delay"
      type="number"
      id="flight-delay"
    />
  </div>
}
```

Debouncing

Especially in reactive UIs with search filters, debouncing is essential: instead of firing an HTTP request on every keystroke, we trigger it only after the user pauses typing. In our example, this behavior appears in the flight search form.

As already teased in the previous chapter, the form's debouncing behavior is specified in its schema. For this, we added the `debounce` function in the `FlightSearch` component (Listing 2-14).

Listing 2-14

```
// .../feature-booking/flight-search/flight-search.ts

[...]

import { debounce, form, minLength, required } from '@angular/forms/signals';

[...]

protected readonly filterForm = form(this.filter, (path) => {
  debounce(path, 300);

  required(path.from);
  minLength(path.from, 3);
});
```

In cases where you want to debounce until the user left the input field, you can specify `blur` instead of a debounce time:

```
protected readonly filterForm = form(this.filter, (path) => {
  debounce(path, 'blur');
});
```

In most cases, just calling `debounce` with the debounce time in milliseconds is exactly what you need. However, if you want to control the debounce time using code, you can also pass in a custom debouncer function (Listing 2-15).

Listing 2-15

```
protected readonly filterForm = form(this.filter, (path) => {
  debounce(path, (ctx, _abortSignal) => {
    return new Promise((resolve) => {
      setTimeout(resolve, 300);
    });
  });

  required(path.from);
  minLength(path.from, 3);
});
```

Now, the returned Promise controls the debouncing. When it resolves, Angular continues processing the changes.

Validating Against Zod and Standard Schema

Often, there is already something, such as a Zod[1] schema, that describes rules for valid objects. Perhaps you already have them for server-side code, or you generate them from a JSON Schema or an Open API document. Fortunately, you can directly validate against such schemas.

Let's imagine, Zod was already installed in our project (`npm i zod`) and we have this Zod schema for flights (Listing 2-16).

Listing 2-16

```
// src/app/domains/ticketing/data/flight-zod-schema.ts

import { z } from 'zod';

export const FlightZodSchema = z.object({
  id: z.number().int(),
  from: z.string().min(3).max(20),
  to: z.string().min(3).max(20),
  date: z.string(),
  delayed: z.boolean(),
})
```

Now, our Signal Form schema use `validateStandardSchema` to reference the Zod schema and use it to validate the form (Listing 2-17).

Listing 2-17

```
// src/app/domains/ticketing/data/flight-schema.ts

import { validateStandardSchema, schema } from '@angular/forms/signals';
import { FlightZodSchema } from './flight-zod-schema';

[...]

export const flightSchema = schema<Flight>((path) => {
  validateStandardSchema(path, FlightZodSchema);
  // ... other validation rules
});
```

This not only works with Zod but also with all other schema libraries that follow the Standard Schema definition such as Valibot[2].

Combined with the `submit` function discussed above, this is a quick and effective way to establish a client-side validation.

[1] https://zod.dev
[2] https://valibot.dev

Submitting Forms

The perhaps biggest improvement in Signals Forms is about how we submit forms: We can now define the logic for submitting a form when calling the `form` function. After binding the resulting `FieldTree` to the `form` tag, all we need is a regular submit button. This section explains these new capabilities in more detail.

Logic for Submitting Forms

For defining our submission logic, we use the new `submission` node in the `options` object that can be passed to the `form` helper (Listing 2-18).

Listing 2-18

```
// .../ticketing/feature-booking/flight-edit/flight-edit.ts

@Component({
  selector: 'app-flight-edit',
  imports: [
    [...]
    FormRoot,
  ],
  [...]
})
export class FlightEdit {

  [...]

  protected readonly flightForm = form(this.flight, flightSchema, {
    submission: {
      action: async (form) => this.save(form),
      ignoreValidators: 'none',
      onInvalid: (form) => this.reportValidationError(form),
    },
  });

  [...]
}
```

The action property points to the function to be executed when the user submits the form. By default, this method is not executed when there is any failing or pending validator. Pending validators are asynchronous validators that did not deliver a result yet. Using the `ignoreValidators` property, we can, however, change this behavior. It can have one of the following values:

- `none`: Validators are not ignored and the form cannot be submitted when any validator is failing or pending. This is the default behavior.

- `pending`: Pending validators do not prevent submitting the form.
- `all`: All Validators are ignored. Hence, we can submit the form even when validators are failing or pending.

The `onInvalid` handler is executed when a submission is prevented by a failing validator.

The method registered with `action` can return validation error the application gets aware of when sending the data to the backend (Listing 2-19).

Listing 2-19

```
protected async save(form: FieldTree<Flight>) {
  try {
    await this.store.saveFlight(form().value());
    return null;
  } catch (error) {
    return {
      kind: 'processing_error',
      error: extractError(error),
    };
  }
}
```

These validation errors are placed in the object graph describing the Signal Form. Hence, the forms error property will provide the error message:

```
<p>
  {{ flightForm().errorSummary() | json }}
</p>
```

This interplay between validation errors received on the client side and validation errors we got during the submission is a welcoming new feature of Signal Forms. With the previous form implementations, such a behavior was difficult to achieve.

In our example the `onInvalid` handler is calling the `reportValidationError` that is displaying a snack bar and focussing the first input with a validation error (Listing 2-20).

Listing 2-20

```
private reportValidationError(form: FieldTree<Flight>): void {
  this.snackBar.open('Please correct the validation errors', 'OK');
  this.focusInvalid(form);
}

private focusInvalid(form: FieldTree<Flight>) {
  const errors = form().errorSummary();
  if (errors.length > 0) {
    errors[0].fieldTree().focusBoundControl();
  }
}
```

Template for Submitting

To submit the form, we just need to connect the form element to our Signal Form. For this, we use the newly introduced `formRoot` directive (Listing 2-21).

Listing 2-21

```
<!-- .../ticketing/feature-booking/flight-edit/flight-edit.html -->

<h1>Flight Edit</h1>

<form [formRoot]="flightForm">

  [...]

  <div>
    <button>Save</button>
  </div>
</form>
```

This new directive takes care of several tasks at once:

- Disabling at default submission behavior, as we don't want to post back the form's content to the server as in traditional server-side rendered applications.
- Disabling the browser's built-in form validation, as Angular is taking care of this and we don't want to duplicate some of the validation messages.
- Connecting the submission action to the form.

As `formRoot` directly registeres the defined `action` with the form's `submit` event, we just need to add an ordinary button with the type `submit` to the form. As `submit` is the default type, we don't even need to add `type="submit"` as shown in the example above. But also when submitting the form by pressing enter, the `action` will be triggered.

Further Submit Actions

If you have further submission actions, e.g., for handing in changes for approval, you can always add further buttons with `type="button"` and bind them to custom click handlers. In such cases, the `submit` function provided by Signal Forms helps to only execute the submission logic when the form is valid (Listing 2-22).

Listing 2-22

```
protected async requestApproval(): Promise<void> {
  await submit(this.flightForm, {
    action: async (form) => {
      await this.store.requestApproval(form().value());
    },
    ignoreValidators: 'none',
    onInvalid: (form) => this.reportValidationError(form),
  });
}
```

Summary

Signal Forms introduce a signal-based model for building and validating forms in Angular. Form state, values, and validation results are represented as signals, making them fully reactive and composable. This unifies user interaction and application state under a single set of reactive primitives.

3 State Management with NgRx Signal Store

Unlike classic multi-page web apps, Single Page Applications keep state while the user switches between routes. This state results from data loaded via HTTP but also from UI interactions, e.g., defined filters or selected items. This shows that we need to manage state in a structured way.

However, if we implement state management manually, we end up writing a lot of repetitive code. This includes tasks such as exposing read-only signals that are writable inside the store or integrating resources for loading data. NgRx SignalStore removes much of this boilerplate and provides a powerful way to manage state. It is the most popular state management solution for modern Angular applications and integrates seamlessly with Angular's signal-based reactivity model.

Creating a Signal Store

In this section, we create our first SignalStore for loading flights. Technically, such a store is a service that holds state for a specific feature of the application.

To install the SignalStore, you just need to add the package `@ngrx/signals` to your application:

```
npm i @ngrx/signals
```

After that, we can create our first store. In NgRx SignalStore, a store is an Angular service created with `signalStore` (Listing 3-1).

Listing 3-1

```
// src/app/domains/ticketing/feature-booking/flight-search/flight-store.ts

import { computed, inject } from '@angular/core';
import { withResource } from '@angular-architects/ngrx-toolkit';
import {
  patchState,
  signalStore,
  withComputed,
  withMethods,
  withProps,
  withState,
} from '@ngrx/signals';

import { Flight } from '../../data/flight';
import { FlightClient } from '../../data/flight-client';

export const FlightStore = signalStore(
  { providedIn: 'root' },

  withState({
    from: 'Graz',
    to: 'Hamburg',
    basket: {} as Record<number, boolean>,
    delayInMin: 0,
  }),

  withProps([...]),
  withResource([...]),
  withComputed([...]),
  withMethods([...]),
  withHooks([...]),
);
```

The first argument can be used to provide the store at the root level. If you want to provide the store by hand, e.g., on a component or route level, you can omit this argument. All other arguments are so-called features that add specific functionality to the store.

Usually, the first feature you add is `withState`, which defines the store's state properties. Each property becomes a Signal that is read-only for consumers but can be updated from within the store.

As TypeScript cannot infer the type of the `basket` property from its initial value, we need to add an explicit type assertion. An alternative to this is explicitly typing the state object (Listing 3-2).

Listing 3-2

```
export interface FlightSearchState {
  from: string;
  to: string;
  basket: Record<number, boolean>;
  delayInMin: number;
}

[...]

withState<FlightSearchState>({
  from: 'Graz',
  to: 'Hamburg',
  basket: {},
  delayInMin: 0,
}),
```

As this is more verbose, I usually go with the first option when I don't need a separate type for the state elsewhere.

Other features used here are `withComputed` for adding computed signals and `withMethods` for defining methods that can update the state or perform side effects. The `withProps` feature allows defining any properties on the store, e.g., to reference injected services.

One of the greatest strengths of the NgRx Signal Store and its underlying design is that you can easily implement features covering code you need time and again. In the example shown, `withResource` is a community extension provided by the NgRx Toolkit (`@angular-architects/ngrx-toolkit`) that integrates Angular's Resource API into the store.

In the following sections, we will look at the individual features in more detail and see how they work together.

Defining State

Using `withProps`, you can add any properties to the store. In our example, we add a property `_flightClient` that points to the injected `FlightClient` service:

```
withProps(() => ({
  _flightClient: inject(FlightClient),
})),
```

The underscore indicates that this is a private member that cannot be used by consumers of the store. This is not just a convention: the NgRx team is really skilled at using TypeScript's type system. In this case, the type system is told not to include

members that start with an underscore in the type exposed to consumers. The use of the underscore is not limited to `withProps` but works for all features that add members.

Further use cases for `withProps` include adding properties that are not signals, such as Observables, for representing events. Like most features, `withProps` takes a function that receives the store and returns the properties to be added. The argument could be used as follows (Listing 3-3).

Listing 3-3

```
withProps((store) => ({
  _filterChanged: new Subject<FlightFilter>({
    from: store.from(),
    to: store.to()
  }),
})),
withProps((store) => ({
  filterChanged: store._filterChanged.asObservable(),
})),
```

This example also shows that the same feature can be used several times. This can be handy when defining properties that depend on other properties. An alternative to this is passing a function with an explicit return (Listing 3-4).

Listing 3-4

```
withProps((store) => {
  const _filterChanged = new Subject<FlightFilter>({
    from: store.from(),
    to: store.to()
  });

  const filterChanged = _filterChanged.asObservable();

  return {
    _filterChanged,
    filterChanged
  };
}),
```

Besides implementing eventing manually, you could also create a custom feature that handles this. We will discuss custom features in a later section. Also, you will see that the Signal Store ships with an elaborate Event API that implements the Flux pattern.

Providing Resources

The `withResource` feature allows you to easily integrate Angular's Resource API into the store. Also, here we pass a function that projects the store into an object with properties added to the store. These properties are resources (Listing 3-5).

Listing 3-5

```
withResource(
  (store) => ({
    flights: store._flightClient.findResource(store.from, store.to),
  }),
  { errorHandling: 'previous value' },
),
```

It does not matter which concrete resource implementation you use, such as `httpResource` or `rxResource`. In the shown case, the resource is created by the `FlightClient` service. Its `findResource` method takes Signals for `from` and `to`, and whenever these change, the resource is triggered again.

The options object passed as the second parameter allows you to configure the error-handling behavior. By default, when a resource is in an error state, reading its value throws an error. This well-intentioned behavior prevents errors from going unnoticed, but it can be inconvenient in some cases. For this reason, `errorHandling` allows us to change this behavior:

`previous value`
: When an error occurs, the resource keeps its previous value and does not throw the error.

`undefined value`
: When an error occurs, the value becomes `undefined` and does not throw an error. This adds `undefined` to the resource's type.

`native`
: Angular's native behavior: when an error occurs, reading the value throws the error.

When adding a resource with `withResource`, the store receives several properties. The resource name is used as a prefix. In our case, we get:

`flightsValue`
: A signal with the value provided by the resource

`flightsIsLoading`
: A signal with a boolean indicating whether the resource is currently loading

`flightsError`
: A signal with an error object, if the resource is in an error state

_flightsReload
: A method to trigger the resource again

flightsStatus
: A signal with the current status of the resource (e.g., 'idle', 'loading', 'error', 'resolved')

flightsHasValue
: A method that returns `true` if the resource has a value (e.g., if it is not loading and not in an error state)

Providing Computed Signals

With `withComputed`, you can add computed signals to the store. These are derived from other signals and automatically update when their dependencies change (Listing 3-6).

Listing 3-6

```
withComputed((store) => ({
  flightsWithDelays: computed(() =>
    toFlightsWithDelays(store.flightsValue(), store.delayInMin()),
  ),
})),
```

The function `toFlightsWithDelays` is a pure function that calculates the new flight times based on the flights loaded via the resource and the `delay` (Listing 3-7).

Listing 3-7

```
function toFlightsWithDelays(flights: Flight[], delay: number): Flight[] {
  if (flights.length === 0) {
    return [];
  }

  const ONE_MINUTE = 1000 * 60;
  const oldFlights = flights;
  const oldFlight = oldFlights[0];
  const oldDate = new Date(oldFlight.date);
  const newDate = new Date(oldDate.getTime() + delay * ONE_MINUTE);
  const newFlight = { ...oldFlight, date: newDate.toISOString() };

  return [newFlight, ...flights.slice(1)];
}
```

Computed signals are the counterpart to selectors in the traditional NgRx Store. They are often used to provide the state in a form needed by the component. These objects are also called View Models.

Providing Methods

Stores make sure the managed state can only be modified in a controlled way. For these updates, you can provide methods with `withMethods` (Listing 3-8).

Listing 3-8

```
withMethods((store) => ({
  updateFilter(from: string, to: string): void {
    patchState(store, { from, to });
  },

  updateBasket(flightId: number, selected: boolean): void {
    patchState(store, (state) => ({
      basket: {
        ...state.basket,
        [flightId]: selected,
      },
    }));
  },

  reload(): void {
    store._flightsReload();
  },

  delay(): void {
    patchState(store, (state) => ({
      delayInMin: state.delayInMin + 15,
    }));
  },
})),
```

By default, the function `patchState` can only be called inside the store. It takes a store and either a partial state or an updater function to modify the state. If you pass a partial state, only the provided properties are updated:

```
patchState(store, { from, to });
```

An updater is a function that maps the entire state to a partial state to be used for the update (Listing 3-9).

Listing 3-9

```
patchState(store, (state) => ({
  basket: {
    ...state.basket,
    [flightId]: selected,
  },
}));
```

This is useful when the new state depends on the old state, e.g., when updating a nested object or array.

Please note that state has to be treated as immutable. In the shown example, we create a new `basket` object instead of modifying the existing one.

It's also interesting to see the interplay of these methods and other features: The `updateFilter` method updates the `from` and `to` state, which triggers the resource to load new flights. The `reload` method directly triggers the resource to load new flights, and the `delay` method updates the delay, which in turn updates the computed `flightsWithDelays`.

Setting up Hooks

Hooks are registered functions that run at specific points in the store's lifecycle: The `onInit` hook runs when the store is initialized and allows you to load initial data. The `onDestroy` hook runs when the store is destroyed and can be used for cleanup tasks. For setting up hooks, the `withHooks` feature is used (Listing 3-10).

Listing 3-10

```
withHooks((store) => ({
  onInit() {
    console.log('FlightStore initialized', store.from(), store.to());
  },
  onDestroy() {
    console.log('FlightStore destroyed', store.from(), store.to());
  },
})),
```

Consuming the Store

Consumers inject the store and read its signals. Updating the state happens by delegating to the store's methods (Listing 3-11).

Listing 3-11

```
// src/app/domains/ticketing/feature-booking/flight-search/flight-search.ts
[...]

import { FlightStore } from './flight-store';

@Component([...])
export class FlightSearch {
  private readonly store = inject(FlightStore);
  private readonly snackBar = inject(MatSnackBar);

  protected readonly filter = linkedSignal(() => ({
    from: this.store.from(),
    to: this.store.to(),
  }));

  protected readonly filterForm = form(this.filter);

  protected readonly flights = this.store.flightsWithDelays;
  protected readonly isLoading = this.store.flightsIsLoading;
  protected readonly error = this.store.flightsError;

  protected readonly basket = this.store.basket;

  [...]

  protected search(): void {
    this.store.updateFilter(this.filter().from, this.filter().to);
  }

  protected updateBasket(flightId: number, selected: boolean): void {
    this.store.updateBasket(flightId, selected);
  }

  protected delay(): void {
    this.store.delay();
  }
}
```

Here, we can clearly see the so-called unidirectional data flow: the user's intention is sent upwards to the store via methods such as `updateFilter`. This results in tasks performed within the store, such as loading flights via the resource. Then the result, such as status flags and loaded flights, is sent downward to the component via signals like `flightsWithDelays` and `flightsIsLoading`. Finally, they are consumed there, which, in most cases, happens via data binding in the template.

We also see some of the strongly typed properties `withResource` added to the store for the flights resource (`flightsIsLoading`, `flightsError`) and the computed `flightsWithDelays` that is derived from the resource and the delay.

Inspecting the Store with the Redux DevTools

One of the most helpful features of stores in general is the possibility to inspect state changes when debugging your application. This section shows how to do this with the Redux DevTools.

Connecting to the Redux DevTools

The Redux DevTools is a browser extension for Chrome and Firefox that lets you inspect your application's state and how it changes over time. It is especially useful for debugging state management solutions. After installing it, you find a new tab in your browser's developer tools called "Redux".

Even though SignalStore doesn't implement the Redux pattern, you can still connect it to the Redux DevTools. For this, just add the `withDevtools` feature provided by the NgRx Toolkit to your store (Listing 3-12).

Listing 3-12

```
import { withDevtools } from '@angular-architects/ngrx-toolkit';

[...]

export const FlightStore = signalStore(
  { providedIn: 'root' },

  [...]

  withDevtools('flight'),
);
```

The passed string is the name of a branch added to the DevTools state tree. This allows you to distinguish between different stores in the DevTools. When you open the DevTools at runtime, you can see your current values (Figure 3-1).

The list on the left shows all recorded state changes. When you click on one of them, you can see the state after the change on the right. Also, the bottom slider lets you go back and forth between state changes. This extremely helpful feature is known as time-travel debugging.

Disabling DevTools in Production

As we don't want to turn on DevTools in production, we need to make sure to only add the `withDevtools` feature in development. An easy way to achieve this is using Angular's `isDevMode` function (Listing 3-13).

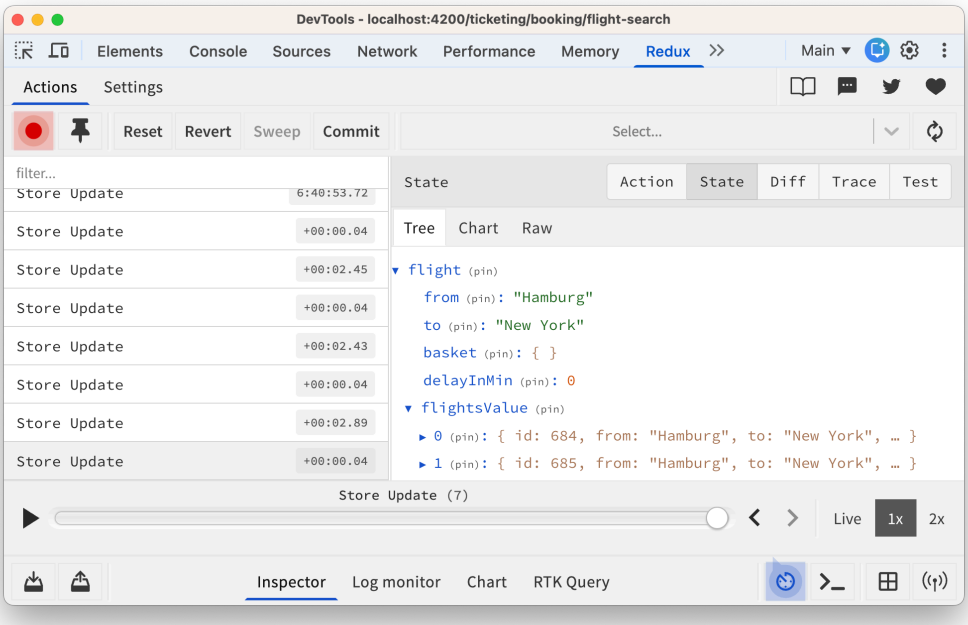


Figure 3-1: Redux DevTools showing the current state of the store

Listing 3-13

```
import { isDevMode } from '@angular/core';
import {
  withDevtools,
  withDevToolsStub
} from '@angular-architects/ngrx-toolkit';

[...]

export const FlightStore = signalStore(
  { providedIn: 'root' },

  [...]

  isDevMode() ? withDevtools('flight') : withDevToolsStub('flight'),
);
```

Here, only in development mode is the real `withDevtools` added. In production, a stub is used that does nothing but has the same API. While this works, it has a drawback: since the `isDevMode` check is only performed at runtime, the DevTools cannot be removed from the production bundle.

If we want to ensure that DevTools are not included in the production bundle, we need a compile-time check. For this, the check needs to use a constant value. To

obtain such a constant, we can introduce an environment file that is replaced when the application starts for development. For this, we run the following schematic:

```
ng generate environments
```

This gives us two files: `environment.ts` and `environment.development.ts`. In production, the first one is used. When started for development, the Angular CLI replaces the first one with the second one. Now, we can add a `withDevtools` method to these files (Listing 3-14).

Listing 3-14

```
// src/environments/environment.ts
import { withDevtools } from '@angular-architects/ngrx-toolkit';

export const environment = {
  withDevtools: withDevtools
};

// src/environments/environment.development.ts
import { withDevToolsStub } from '@angular-architects/ngrx-toolkit';

export const environment = {
  withDevtools: withDevToolsStub
};
```

To avoid importing the wrong environment file, we can create a helper function that calls the `withDevtools` method from the `environment` (Listing 3-15).

Listing 3-15

```
// src/app/domains/shared/util-common/with-dev-tools-for-debug-mode.ts

import { environment } from "../../../../environments/environment";

export function withDevToolsForDebugMode(name: string) {
  return environment.withDevtools(name);
}
```

Finally, the new helper function is called in the store instead of `withDevtools` (Listing 3-16).

Listing 3-16

```
export const FlightStore = signalStore(
  { providedIn: 'root' },

  [...]
  withDevToolsForDebugMode('flight')
);
```

Mutations

While resources are great for loading data, unfortunately, Angular does not provide a counterpart for modifying data. We could do this manually by providing methods that use the `HttpClient` directly, or by providing a service that delegates to the `HttpClient`. However, this would result in a lot of repetitive code, e.g., for setting the loading and error state, handling overlapping calls, or for integrating with the store's state. The NgRx Toolkit provides a solution for this: the Mutation API.

This mutation API is inspired by open-source solutions such as React Query and a proof-of-concept by Marko Stanimirović, the lead developer of SignalStore. Its use feels familiar as it mirrors the Resource API. Currently, there are `rxMutation` for arbitrary changes that return Observables, and `httpMutation` for changes that can be represented directly as HTTP requests.

This chapter shows how to use the Mutation API and integrate it with the store. For this, we look at the `FlightDetailStore` used in our example application for flight editing.

Creating Mutations

For adding mutations to the store, you can use the `withMutations` feature (Listing 3-17).

Listing 3-17

```
// src/app/domains/ticketing/feature-booking/flight-edit/flight-detail-store.ts

[...]

withMutations((store) => ({
  saveFlight: httpMutation<Flight, Flight>({
    request: (flight: Flight) => ({
      url: `[...]/flight/${flight.id}`,
      method: 'PUT',
      body: flight,
    }),
    operator: concatOp,
    onSuccess(result, param) {
      console.log('Flight sent to the server', param);
      console.log('Flight the server sent back', result);

      store._snackBar.open('Flight updated successfully', 'OK', {
        duration: 3000,
      });
    },
    onError(error, param) {
      console.log('Flight sent to the server', param);
      console.log('Error the server sent back', error);

      store._snackBar.open('Failed to update flight', 'OK', {
        duration: 5000,
      });
    },
  }),
})),

[...]
```

Similar to `withResource`, `withMutations` does not care which kind of mutation we are adding as long as it fulfils the Mutation API. Here, we go with an `httpMutation` but we could also use an `rxMutation` that returns an Observable.

The two type parameters define the type of the passed argument and the type of the returned value. In our case, both are `Flight` – the flight to be saved and the saved flight returned by the server. As a result of this typing, the store will get a `saveFlight` method that accepts a flight, triggers the mutation, and returns the saved flight.

The request method we pass to `httpMutation` defines the HTTP request for the mutation. In the shown example, it is a PUT call that includes the flight ID in the URL and the entire flight in the body. The mutation will also add status properties such as `saveFlightIsPending` and `saveFlightError` to the store. This all happens automatically without the need to write any boilerplate code.

The property `operator` defines the semantics for overlapping calls. The possible values mirror the behavior of the RxJS flattening operators:

`switchOp` : Cancels the previous call when a new one starts. Use it when only the latest request matters.

`mergeOp` : Runs all calls in parallel and lets them complete independently. This is useful when every request should be processed, even if they overlap.

`concatOp` : Queues calls and runs them one after the other.

`exhaustOp` : Ignores new calls while a call is still running. Use it to prevent accidental double submits.

When no operator is specified, `concatOp` is used by default. This is a safe choice for many use cases, as it preserves the order of requests, prevents overlapping requests and the resulting race conditions, and does not cancel any requests.

When the operation succeeds, the `onSuccess` handler logs the sent and received flight and displays a snackbar. The needed SnackBar service comes from Angular Material and can be injected using `withProps`. The `onError` handler proceeds analogously but logs the received error instead of the result.

Of course, usually we don't want to have all these protocol details in the store. Hence, analogous to the resource, it's a good idea to create the mutation in our `FlightClient` (Listing 3-18).

Listing 3-18

```
// src/app/domains/ticketing/data/flight-client.ts
import { httpMutation } from '@angular-architects/ngrx-toolkit';

[…]

@Injectable({
  providedIn: 'root',
})
export class FlightClient {

  [...]

  createSaveMutation(options: Partial<HttpMutationOptions<Flight, Flight>>) {
    return httpMutation({
      ...options,
      request: (flight: Flight) => ({
        url: `${this.configService.baseUrl}/flight/${flight.id}`,
        method: 'PUT',
        body: flight,
        headers: {
          Accept: 'application/json',
        },
```

```
      }),
      operator: concatOp,
    });
  }
}
```

As the store needs to define the `onSuccess` and `onError` callbacks, `createSaveMutation` takes an options object with these callbacks as a parameter and spreads it to the `httpMutation`. The store can now delegate to the `FlightClient` for creating the mutation (Listing 3-19).

Listing 3-19

```
withMutations((store) => ({
  saveFlight: store._flightClient.createSaveMutation({
    onSuccess(result, param) {
      console.log('Flight sent to the server', param);
      console.log('Flight the server sent back', result);

      store._snackBar.open('Flight updated successfully', 'OK', {
        duration: 3000,
      });
    },
    onError(error, param) {
      console.log('Flight sent to the server', param);
      console.log('Error the server sent back', error);

      store._snackBar.open('Failed to update flight', 'OK', {
        duration: 5000,
      });
    },
  }),
})),
```

Consuming Mutations

Because we introduced a mutation called `saveFlight`, the store now has a method `saveFlight` but also status signals such as `saveFlightIsPending` and `saveFlightError`. We can use them in our `FlightEdit` component (Listing 3-20).

Listing 3-20

```
// src/app/domains/ticketing/feature-booking/flight-edit/flight-edit.ts
[...]

import { FlightDetailStore } from './flight-detail-store';

@Component([...])
export class FlightEdit {
  private readonly store = inject(FlightDetailStore);

  [...]

  protected readonly flight = linkedSignal(() =>
    normalizeFlight(this.store.flightValue()),
  );

  // Properties added by withMutations
  protected readonly isPending = this.store.saveFlightIsPending;
  protected readonly error = this.store.saveFlightError;

  protected readonly flightForm = form(this.flight);

  protected readonly isDisabled = computed(
    () => this.flightForm().invalid() || this.isPending(),
  );

  [...]

  protected async save(): Promise<void> {
     const result = await this.store.saveFlight(this.flight());

    if (result.status === 'success') {
      console.log('Flight saved successfully', result.value);
    } else if (result.status === 'error') {
      console.error('Failed to save flight', result.error);
    }
    else {
      console.warn('Mutation was cancelled');
    }
  }
}

function normalizeFlight(flight: Flight): Flight {
  const localDate = flight.date.substring(0, 16);
  return {
    ...flight,
    date: localDate,
  };
}
```

Once again, we use a `linkedSignal` to provide a writable signal for the Signal Form. With `normalizeFlight`, we update the date format so it can be used directly with an `<input type="datetime-local">` for simplicity.

The `saveFlight` method accepts a `Flight` and returns a promise with the result of the mutation. This result can be in one of three states: `success`, `error`, or `cancelled`. The latter one might surprise you at first, but it is a consequence of the fact that mutations can be cancelled when we use `switchOp` or `exhaustOp`.

In the case of `success`, the result contains the saved flight. In the `error` state, we get an error object.

Using rxMutation

Besides an `httpMutation`, the Toolkit also provides an `rxMutation` that can delegate to arbitrary mutations returning Observables. The usage is very similar to `httpMutation` but instead of a `request` method, you need to provide an `operation` (Listing 3-21).

Listing 3-21

```
[...]

createSaveRxMutation(options: Partial<RxMutationOptions<Flight, Flight>>) {
  return rxMutation({
    ...options,
    operation: (flight: Flight) => this.update(flight),
    operator: concatOp,
  });
}

[...]
```

This implementation delegates to the `FlightClient`'s `update` method, which performs the HTTP request in a traditional way using the `HttpClient` and returns an Observable (Listing 3-22).

Listing 3-22

```
// src/app/domains/ticketing/data/flight-client.ts

[...]

update(flight: Flight): Observable<Flight> {
  const url = `${this.configService.baseUrl}/flight/${flight.id}`;

  const headers = {
    Accept: 'application/json',
  };

  return this.http.put<Flight>(url, flight, { headers });
}

[...]
```

From the store's and component's perspective, there is no difference between `httpMutation` and `rxMutation`: Both add the same method and status signals to the store.

Summary

NgRx SignalStore models feature state as signals and clearly separates reads and writes. Components consume a stable, read-focused surface (including derived view models), while all state changes and data loading stay encapsulated behind an explicit store API.

4 Signal Queries

Modern Angular applications and component libraries consist of many components that need to work together. This chapter introduces Signal Queries and component composition. You will learn how to write reusable components that can be extended by child components passed by the caller. You will also see options for communication between such components.

Content Projection

Content Projection allows a component to receive markup, such as HTML, from the caller. It can display this in its own template.

To illustrate this concept, we use a simple tab control (Figure 4-1).

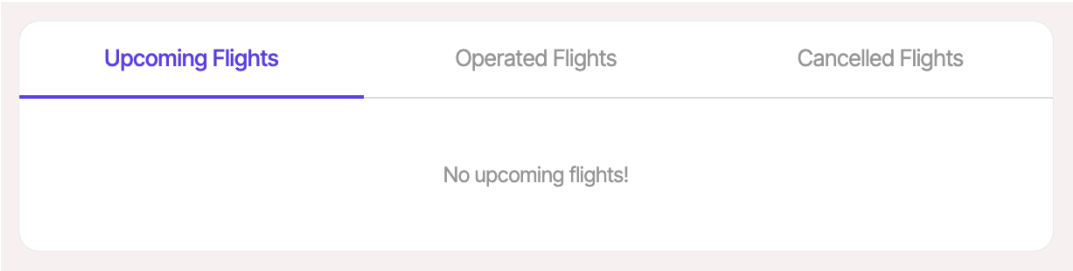


Figure 4-1: Tab control

The links at the top allow switching between the individual tabs. The entire tab control is implemented by a component that we will refer to as `TabbedPane`, and a `Tab` component is used for each individual tab.

In the flights42 sample application, the tabbed pane is implemented under `src/app/domains/shared/ui-common/` in three variants: injection (parent reference), query (`contentChildren`), and service (`TabRegistry`). You can generate similar components with the CLI:

```
ng g c shared/ui-common/tabbed-pane
ng g c shared/ui-common/tab
```

Our `Tab` component needs this behavior because it must present the passed elements within the tab it displays:

```
<app-tab title="Upcoming Flights">
   <p>No upcoming flights!</p>
</app-tab>
```

To show how this is possible, we take a look at the component's implementation. The component class (Listing 4-1).

Listing 4-1

```
// src/app/domains/shared/ui-common/injection-tabbed-pane/tab.ts

import { Component, computed, inject, input } from '@angular/core';

import { TabbedPane } from './tabbed-pane';

@Component({
  selector: 'app-tab',
  imports: [],
  template: `
    @if (visible()) {
      <div class="tab-content">
        <ng-content></ng-content>
      </div>
    }
  `,
  styles: `[...]`,
})
export class Tab {
  // private pane = inject(TabbedPane, { optional: true });
  private pane = inject(TabbedPane);
  readonly title = input.required<string>();

  protected readonly visible = computed(() => this.pane.currentTab() === this);

  constructor() {
    this.pane.registerTab(this);
  }
}
```

The implementation uses an `input` for the title and a computed `visible` signal that indicates whether this tab is the current one (we will see how the parent `TabbedPane` coordinates this in the next section). The template uses `@if (visible())` and `ng-content` to display the projected content in the inline template below.

To specify where this content is to be displayed within the tab, the component marks the desired location with the `ng-content` element.

To try out the new component, we can call it in the About page template (Listing 4-2).

Listing 4-2

```
<!-- src/app/shell/about/about.html -->

<h1>About</h1>

<app-tabbed-pane>
    <app-tab title="Upcoming Flights">
        <p>No upcoming flights!</p>
    </app-tab>

    <app-tab title="Operated Flights">
        <p>No operated flights!</p>
    </app-tab>

    <app-tab title="Cancelled Flights">
        <p>No cancelled flights!</p>
    </app-tab>
</app-tabbed-pane>
```

So that only the active tab is displayed at a time, the `TabbedPane` wraps the tabs and coordinates which one is visible. We introduce `TabbedPane` in the next section.

Referencing Parent Components

The `TabbedPane` is to receive the `Tab` components to be displayed via content projection when called (Listing 4-3).

Listing 4-3

```
<app-tabbed-pane>
    <app-tab title="Upcoming Flights">
        <p>No upcoming flights!</p>
    </app-tab>

    <app-tab title="Operated Flights">
        <p>No operated flights!</p>
    </app-tab>

    <app-tab title="Cancelled Flights">
        <p>No cancelled flights!</p>
    </app-tab>
</app-tabbed-pane>
```

It manages these tabs in a `tabs` signal. Each tab registers itself via `registerTab` when it is created (Listing 4-4).

Listing 4-4

```
// src/app/domains/shared/ui-common/injection-tabbed-pane/tabbed-pane.ts

import { Component, computed, model, signal } from '@angular/core';

import { Tab } from './tab';

@Component({
  selector: 'app-tabbed-pane',
  standalone: true,
  imports: [],
  template: `
    <div class="pane">
      <div class="tabs-header" role="group">
        @for (tab of tabs(); track tab) {
          <button
            class="tab-button"
            [class.active]="tab === currentTab()"
            (click)="activate($index)">
            {{ tab.title() }}
          </button>
        }
      </div>
      <div class="tabs-content">
        <ng-content></ng-content>
      </div>
    </div>
  `,
  styles: `[...]`,
})
export class TabbedPane {
  protected readonly current = model(0);
  protected readonly tabs = signal<Tab[]>([]);

  readonly currentTab = computed(() => this.tabs()[this.current()]);

  registerTab(tab: Tab): void {
    this.tabs.update((tabs) => [...tabs, tab]);
  }

  activate(tabIndex: number): void {
    this.current.set(tabIndex);
  }
}
```

The `current` model holds the index of the active tab; `currentTab` is a computed signal that returns the active tab instance. Tabs register themselves by calling `registerTab` on the parent—the `Tab` component does this in its constructor by injecting `TabbedPane` (as already shown above).

The template uses `@for` to render a button per tab and `ng-content` to project the tab contents. Only the tab for which `visible()` is true (controlled by each `Tab`'s computed signal) renders its content.

To register with the injected parent, the `Tab` passes itself (`this`) to `registerTab` in its constructor.

Note

To ensure that the `Tab` also works without a parent `TabbedPane`, inject with `optional: true` (Listing 4-5).

Listing 4-5

```
private pane = inject(TabbedPane, { optional: true });

constructor() {
  this.pane?.registerTab(this);
}
```

If Angular cannot find an implementation for the token, it returns `undefined`.

View and Content

Each component can have not only a View but also so-called Content. Since distinguishing these two concepts isn't entirely easy, Figure 4-2 illustrates the difference using the `Tab` component we looked at earlier (Figure 4-2).

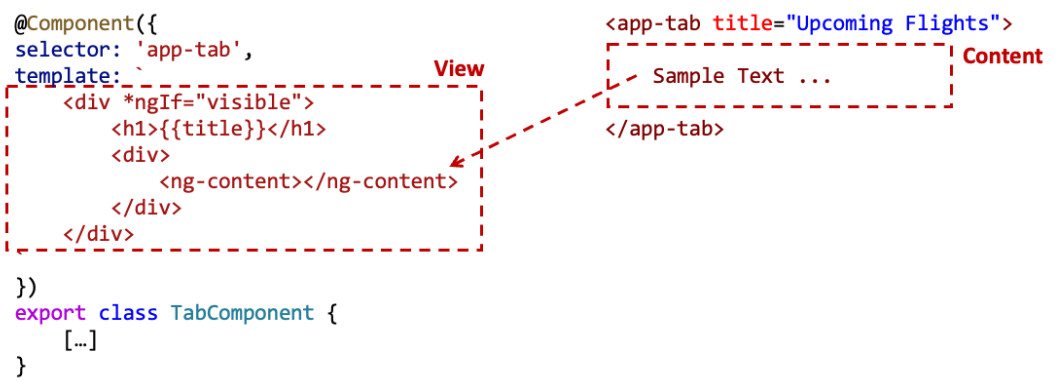


Figure 4-2: Difference between View and Content

For clarity, the templates were included directly as strings in the components here.

While the View is defined by the template and thus determines the component's presentation at runtime, Content is the markup that the caller passes to the component. As we mentioned above, Angular places this markup in the template's `ng-content` element via Content Projection.

Interacting with Content

A component can directly access its View and Content and query individual components placed in them. To illustrate accessing content, we use a second variant of the tabbed pane in which the pane queries its tabs rather than having them register.

Here, the `TabbedPane` gets a reference to its `Tab` components by querying the content using `contentChildren` (Listing 4-6).

Listing 4-6

```
// src/app/domains/shared/ui-common/query-tabbed-pane/tabbed-pane.ts

import {
  Component,
  computed,
  contentChildren,
  effect,
  model,
} from '@angular/core';

import { Tab } from './tab';

@Component({
  selector: 'app-tabbed-pane',
  standalone: true,
  imports: [],
  template: `
    <div class="pane">
      <div class="tabs-header" role="group">
        @for (tab of tabs(); track tab) {
          <button
            class="tab-button"
            [class.active]="tab === currentTab()"
            (click)="activate($index)">
            {{ tab.title() }}
          </button>
        }
      </div>
      <div class="tabs-content">
        <ng-content></ng-content>
      </div>
    </div>
  `,
})
export class TabbedPane {
  protected readonly current = model(0);
  protected readonly tabs = contentChildren(Tab);
```

```
  readonly currentTab = computed(() => this.tabs()[this.current()]);

  constructor() {
    effect(() => {
      for (const tab of this.tabs()) {
        tab.pane = this;
      }
    });
  }

  activate(tabIndex: number): void {
    this.current.set(tabIndex);
  }
}
```

For direct access to the Content and the tabs positioned in it, the component defines a `tabs` property using the `contentChildren` query. It receives the `Tab` type as a filter and returns a signal whose value is the array of projected `Tab` instances.

The `Tab` in this variant does not inject the parent; instead, it exposes a setter `pane` that the pane's effect calls, so each tab knows its parent (Listing 4-7).

Listing 4-7

```
// src/app/domains/shared/ui-common/query-tabbed-pane/tab.ts

import { Component, computed, input } from '@angular/core';

import { TabbedPane } from './tabbed-pane';

@Component({
  selector: 'app-tab',
  standalone: true,
  imports: [],
  template: `
    @if (visible()) {
      <div class="tab-content">
        <ng-content></ng-content>
      </div>
    }
  `,
  [...]
})
export class Tab {
  private _pane?: TabbedPane;
  readonly title = input.required<string>();

  protected readonly visible = computed(
    () => this._pane?.currentTab() === this,
```

```
  );

  set pane(pane: TabbedPane) {
    this._pane = pane;
  }
}
```

In addition to `contentChildren`, `contentChild` exists for querying exactly one element from the content:

```
readonly firstTab = contentChild(Tab);
```

Both can also use a template reference as a filter:

```
readonly tabs = contentChildren<Tab>('tab');
```

The Content could look like this:

```
<app-tab title="A" #tab></app-tab>
<app-tab title="B" #tab></app-tab>
```

Similarly, `contentChild` can query a single element with that template reference:

```
readonly firstTab = contentChild('tab');
```

Interacting with the View

Just as a component can query its Content with `contentChild` and `contentChildren`, it can use `viewChild` and `viewChildren` to query its View. Both return signals that update when the queried elements are available. Analogously to `contentChild` (one element) and `contentChildren` (several), `viewChild` refers to a single element in the view, while `viewChildren` lets you reference multiple elements.

An example of using viewChild can be found in our example application's `ReportPage` component. This component draws a diagram using the popular Chart.js library. Chart.js needs a reference to a canvas element in the template. To get this reference, the ReportPage uses `viewChild` (Listing 4-8).

Listing 4-8

```
// src/app/domains/ticketing/feature-reporting/reporting-page/reporting-page.ts

import {
  afterRenderEffect,
  Component,
  ElementRef,
  signal,
  viewChild,
} from '@angular/core';

[...]

@Component({
  selector: 'app-reporting-page',
  imports: [...],
  templateUrl: './reporting-page.html',
  styleUrl: './reporting-page.css',
})
export class ReportingPage {
  readonly canvas = viewChild<ElementRef<HTMLCanvasElement>>('chart');

  protected readonly data = signal<DataItem[]>([]);
  [...]

  constructor() {
    afterRenderEffect(() => {
      const data = this.data();
      const canvasElm = this.canvas();
      const canvas = canvasElm?.nativeElement;

      if (canvas) {
        this.renderChart(data, canvas);
      }
    });
  }

  private renderChart(data: DataItem[], canvas: HTMLCanvasElement) {
    const chart = new Chart(canvas, { [...]});
    chart.render();
  }
}
```

The template assigns the template reference `#chart` to the canvas (Listing 4-9).

Listing 4-9

```
<!-- .../ticketing/feature-reporting/reporting-page/reporting-page.html -->

<div class="chart-wrapper">
  <canvas #chart></canvas>
</div>
```

The component uses `afterRenderEffect` to ensure the view is ready and the canvas has already been rendered.

Questioning the Use of viewChild and viewChildren

As became clear in the last section, conventional data binding can be replaced by direct interaction with the View. `viewChild` and `viewChildren` allow querying individual elements of the View.

In most cases, however, this is not a good idea! With this approach, you replace a core Angular feature – declarative data binding – with your own code. This makes the code harder to follow and maintain. Moreover, direct property updates can lead to change-detection cycles.

We question the use of all `viewChild` and `viewChildren` calls in our code reviews. We consider their use justified only when a third-party control does not allow data binding for certain properties when you need the DOM (e.g., for Chart.js), or when a parent must call a method of a child component.

Static Child Components

The signal-based query functions `viewChild`, `viewChildren`, `contentChild`, and `contentChildren` accept an options object as the second argument. The `read` option lets you obtain a different token (e.g., `ElementRef`). The `static` option specifies whether the queried elements are present before or after change detection:

```
readonly chart = viewChild<ElementRef<HTMLCanvasElement>>('chart',
  { static: true });
```

If the elements exist from the start – e.g., they are not inside `@if` or `@for`—you can set `static: true`. Then the signal can have a value earlier.

Otherwise, content children are available after the content has been checked; view children after the view has been initialized. Use `afterRenderEffect` or `afterNextRender` when you need to react once the queried elements are set.

Summary

Content Projection uses `ng-content` to display markup passed from the caller. The View is the component's template; the Content is the markup projected into it. Angular provides signal-based queries—`viewChild`, `viewChildren`, `contentChild`, and `contentChildren`—to access elements in the View or Content.

Using `viewChild` and `viewChildren` is justified only when DOM access is required (e.g., Chart.js), third-party components lack data binding, or a parent must call a child method. For most cases, declarative data binding remains the preferred approach.

Going Further: Modern Angular

If you would like to dive deeper after this compact introduction, our comprehensive book **Modern Angular: Architecture, Concepts, Implementation**[1] is the natural next step:

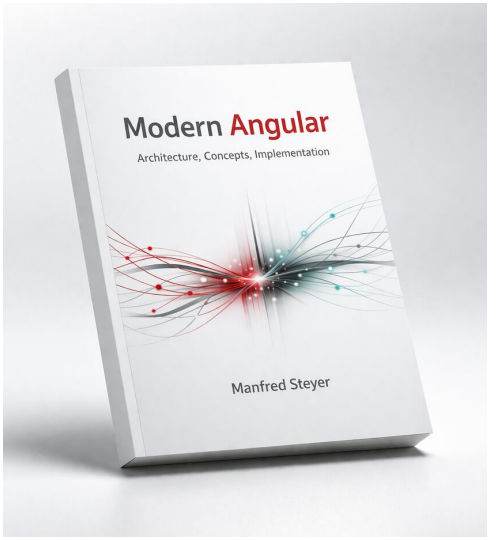


Figure 4-3: Modern Angular: Architecture, Concepts, Implementation

Across roughly **420 pages and 19 chapters**, it shows you how to build **scalable Angular applications** using proven architectural patterns – based on the experience from more than **1,200 workshops** with enterprise teams.

The contents go well beyond the basics covered in this introduction:

- Signal-based components and reactive design with Signals
- Signal Forms and modern testing with Vitest
- Sustainable architectures, NgRx Signal Store, and Signal Queries
- Highly reusable components
- Agentic AI in Angular applications
- Monorepos and Micro Frontends
- Defer, SSR, and Hydration
- ViewContainer and TemplateRef

[1] https://angular-book.com/

As a **Living eBook**, the book is continuously updated for new Angular versions and best practices – all updates are included in the purchase price.

You can find more information and a preview at angular-book.com[2].

[2]https://angular-book.com/

Trainings and Consulting

Learn more about architectures for Angular and huge enterprise as well as industrial solution in our advanced workshop[3]:



Figure 4-4: Advanced Angular Workshop

Save your ticket for one of our **remote or on-site** workshops now or request a company workshop (remote or In-House) for you and your team!

Besides this, we provide the following topics as part of our training or consultancy workshops:

- Agentic UI with Angular
- Angular Essentials: Building Blocks and Concepts
- Modern Angular Testing Workshop (Vitest, Playwright, etc.)
- Modern Angular: Best Practices and Patterns

Please find the full list with our offers here[4].

If you like our offer, keep in touch with us so that you don't miss anything.

[3] http://angulararchitects.io/architecture-workshop
[4] https://www.angulararchitects.io/en/angular-workshops/

Imprint & Legal Notice

Softarc Consulting GmbH
Ludersdorf 219 8200 Gleisdorf Austria

E-Mail: manfred.steyer@angulararchitects.io
Website: https://www.angulararchitects.io

Company Register Number: FN 519830 g
Commercial Court: Landesgericht für ZRS Graz
VAT ID: ATU74807015

Managing Director: Manfred Steyer

© Softarc Consulting GmbH. All rights reserved.

The contents of this book have been created with the greatest possible care. However, no guarantee is given for the accuracy, completeness, or timeliness of the information provided.